

1. Towers of Hanoi

Constraint: Moves allowed only $A \leftrightarrow B$ and $B \leftrightarrow C$ (not $A \leftrightarrow C$).

$M(n)$ = moves for $A \rightarrow C$ (non-adjacent)

$M'(n)$ = moves for $A \rightarrow B$ (adjacent)

Algorithm for $A \rightarrow C$:

- | | |
|--|--------------|
| 1. Move $(n - 1)$ disks: $A \rightarrow C$ | $[M(n - 1)]$ |
| 2. Move disk n : $A \rightarrow B$ | $[1]$ |
| 3. Move $(n - 1)$ disks: $C \rightarrow A$ | $[M(n - 1)]$ |
| 4. Move disk n : $B \rightarrow C$ | $[1]$ |
| 5. Move $(n - 1)$ disks: $A \rightarrow C$ | $[M(n - 1)]$ |

$$M(n) = 3M(n - 1) + 2, \quad M(1) = 2$$

$$M(n) = 3^n - 1$$

$$M(1) = 2 \implies 3A - 1 = 2 \implies A = 1$$

$M(n) = 3^n - 1$

2. Partition

(a) **begin**
 PARTITION($A[1..n], p$)
 $k \leftarrow 0$
 for $i \leftarrow 1$ **to** n **do**
 if $A[i] \leq p$ **then then**
 $k \leftarrow k + 1$
 SWAP($A[k], A[i]$)
 return k

Start of each loop iteration:

$$A[1..k] \leq p$$

$$A[k + 1..i - 1] > p$$

When the algorithm ends:

$$A[1..k] \leq p$$

$$A[k + 1..n] > p$$

(b)

Comparisons: n

Swaps: $\leq n$

$T(n) = O(n)$

3. Selection Recurrence

(a) Groups of size 3:

$$\begin{aligned}
 \text{Groups:} & \quad n/3 \\
 \text{Elements} \leq \text{group median:} & \quad 2 \\
 \text{Medians} \leq \text{pivot:} & \quad \geq \frac{1}{2} \cdot \frac{n}{3} = \frac{n}{6} \\
 \text{Guaranteed} \leq \text{pivot:} & \quad 2 \cdot \frac{n}{6} = \frac{n}{3} \\
 \text{Worst-case recurse:} & \quad n - \frac{n}{3} = \frac{2n}{3}
 \end{aligned}$$

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$$

Note: $\frac{1}{3} + \frac{2}{3} = 1 \implies T(n) = \Theta(n \log n)$

(b) Groups of size $2k + 1$:

$$\begin{aligned}
 \text{Groups:} & \quad \frac{n}{2k+1} \\
 \text{Elements} \leq \text{group median:} & \quad k+1 \\
 \text{Medians} \leq \text{pivot:} & \quad \geq \frac{n}{2(2k+1)} \\
 \text{Guaranteed} \leq \text{pivot:} & \quad (k+1) \cdot \frac{n}{2(2k+1)} = \frac{(k+1)n}{2(2k+1)} \\
 \text{Worst-case:} & \quad n - \frac{(k+1)n}{2(2k+1)} = n \cdot \frac{2(2k+1) - (k+1)}{2(2k+1)} \\
 & \quad = n \cdot \frac{3k+1}{2(2k+1)}
 \end{aligned}$$

$$T(n) = T\left(\frac{n}{2k+1}\right) + T\left(\frac{(3k+1)n}{2(2k+1)}\right) + O(n)$$

Linear time condition:

$$\frac{1}{2k+1} + \frac{3k+1}{2(2k+1)} = \frac{3(k+1)}{2(2k+1)} < 1 \iff k > 1$$

4. Intersection Detection

```
begin
  INTERSECTIONDETECT( $I[1..n]$ )
  Sort  $I$  by  $A[i]$ 
   $m \leftarrow B[1]$ 
  for  $i \leftarrow 2$  to  $n$  do
    if  $A[i] < m$  then
      return true
     $m \leftarrow \max(m, B[i])$ 
  return false
```

$$\forall j < i : A[j] \leq A[i]$$

$$\text{overlap} \iff A[i] < B[j]$$

$$T(n) = 2T(n/2) + O(n)$$

$$\text{Sorting: } O(n \log n)$$

$$\text{Scan: } O(n)$$

$$\boxed{T(n) = O(n \log n)}$$

Divide & Conquer:

begin

```

INTERSECTDC( $I[1..n]$ )
if  $n \leq 1$  then
   $\lfloor$  return false
 $h \leftarrow \lfloor n/2 \rfloor$ 
if INTERSECTDC( $I[1..h]$ ) then
   $\lfloor$  return true
if INTERSECTDC( $I[h+1..n]$ ) then
   $\lfloor$  return true
// Both halves sorted by  $A[\cdot]$ 
 $m_L \leftarrow -\infty$ ,
 $m_R \leftarrow -\infty$ 
 $i \leftarrow 1$ ,
 $j \leftarrow h + 1$ 
while  $i \leq h$  and  $j \leq n$  do
  if  $A[i] \leq A[j]$  then
    if  $A[i] < m_R$  then
       $\lfloor$  return true
     $m_L \leftarrow \max(m_L, B[i])$ 
     $i \leftarrow i + 1$ 
  else
    if  $A[j] < m_L$  then
       $\lfloor$  return true
     $m_R \leftarrow \max(m_R, B[j])$ 
     $j \leftarrow j + 1$ 
while  $i \leq h$  do
  if  $A[i] < m_R$  then
     $\lfloor$  return true
   $i \leftarrow i + 1$ 
while  $j \leq n$  do
  if  $A[j] < m_L$  then
     $\lfloor$  return true
   $j \leftarrow j + 1$ 
Merge  $I[1..h]$ ,  $I[h+1..n]$  by  $A[\cdot]$ 
return false

```

Merge-sort style:

recursive calls sort halves
merge detects cross-overlaps

Sorted $\implies \forall j' < i' : A[j'] \leq A[i']$

Overlap $\iff A[i'] < B[j']$

$\therefore$ track $\max B$ per half

Merge step: $O(n)$

$T(n) = 2T(n/2) + O(n)$

$$T(n) = O(n \log n)$$